ROTEIRO-ALUNO

Roteiro do aluno — Minicurso de OpenTofu

Este roteiro acompanha os exemplos do repositório e apresenta o fluxo básico de Infraestrutura como Código (IaC): declarar o estado desejado, gerar um plano, aplicar as alterações e observar o estado gerido pelo OpenTofu.

Antes de começar

Confirme que tem OpenTofu e Docker com Docker Compose instalados. Abra um terminal na raiz do repositório `minicurso-opentofu`.

```
tofu version
docker --version
docker compose version
```

Os módulos 1 a 3 criam ficheiros locais. O Módulo 4 usa o Floci para simular o serviço AWS S3 localmente; não cria recursos numa conta AWS real.

O ciclo de trabalho do OpenTofu

Nos exemplos, estes comandos formam o ciclo principal:

```
tofu init
tofu fmt
tofu validate
tofu plan
tofu apply
tofu output
```

- `init` prepara o diretório e instala os providers declarados.
- `fmt` formata os ficheiros HCL.
- `validate` verifica a configuração.

- `plan` mostra as alterações propostas sem as aplicar.
- `apply` pede confirmação e aplica o plano.
- `output` mostra os valores publicados pela configuração.

Leia o plano antes de confirmar um `apply`. O ficheiro de estado (`.tfstate`) regista os recursos que o OpenTofu gere; não o edite manualmente nem o partilhe como se fosse um ficheiro de configuração.

Módulo 1 — Setup e primeiro recurso local

Objetivo: conhecer um provider e criar o primeiro recurso gerido pelo OpenTofu.

1. Aceda ao módulo e leia `main.tf` e `outputs.tf`:

   ```
   cd modulo-1-setup
   ```

2. Identifique a versão mínima do OpenTofu, o provider `local` e o recurso `local_file.boas_vindas`.

3. Execute o ciclo:

   ```
   tofu init
   tofu fmt
   tofu validate
   tofu plan
   tofu apply
   tofu output caminho_ficheiro
   ```

4. Abra `hello.txt` e compare o conteúdo com a declaração do recurso.

5. Altere a mensagem `content` em `main.tf`. Execute `tofu plan` e observe a mudança proposta; depois aplique e volte a consultar o ficheiro.

6. Para remover o recurso criado:

   ```
   tofu destroy
   ```

Para refletir: o que mudou no plano quando alterou `content`? Qual a diferença entre declarar o resultado desejado e criar o ficheiro manualmente?

Módulo 2 — Sintaxe HCL, variáveis e ciclo de vida

Objetivo: usar variáveis, providers, interpolação, funções e outputs.

1. Aceda ao módulo:

   ```
   cd ../modulo-2-sintaxe-hcl
   ```

2. Leia `providers.tf`, `variables.tf`, `main.tf`, `outputs.tf` e `terraform.tfvars`. Repare que o ficheiro `.tfvars` define `ambiente` como `prod` e substitui o texto base.

3. Execute:

   ```
   tofu init
   tofu fmt
   tofu validate
   tofu plan
   tofu apply
   tofu output
   ```

4. Consulte o ficheiro criado:

   ```
   cat config-prod-*.txt
   ```

   Observe o ambiente em maiúsculas, o texto fornecido e o sufixo gerado pelo provider `random`.

5. Experimente alterar `ambiente` ou `conteudo_base` em `terraform.tfvars`. Execute `tofu plan` antes de aplicar e veja como o nome e o conteúdo do ficheiro são afetados.

6. Limpe os recursos do exercício:

   ```
   tofu destroy
   ```

Para refletir: que valores vieram dos defaults de `variables.tf` e quais foram substituídos por `terraform.tfvars`?

Módulo 3 — Módulos reutilizáveis e ambientes separados

Objetivo: transformar uma configuração em módulo reutilizável e executá-la com valores diferentes em `dev` e `prod`.

1. A partir do diretório do Módulo 2, entre na raiz do Módulo 3:

```
cd ../modulo-3-estado-modulos
```

2. Leia `modules/gerador_app/variables.tf`, `main.tf` e `outputs.tf`. O módulo recebe o nome, a porta e o diretório de saída, cria um manifesto JSON e publica o caminho do ficheiro.

3. Prepare o diretório de saída e execute o ambiente de desenvolvimento:

```
mkdir -p environments/dev/build
cd environments/dev
tofu init
tofu fmt
tofu validate
tofu plan
tofu apply
tofu output
```

4. Examine os manifestos em `build/`. Compare `frontend-web-manifest.json` e `backend-api-manifest.json`: são duas instâncias do mesmo módulo com valores diferentes.

5. Volte à raiz do Módulo 3 e execute `prod` separadamente:

```
cd ../..
mkdir -p environments/prod/build
cd environments/prod
```

```
tofu init
tofu fmt
tofu validate
tofu plan
tofu apply
tofu output
```

6. Compare os manifestos de `dev` e `prod`. Cada diretório de ambiente mantém o seu próprio estado local.
7. Para remover os recursos geridos, execute `tofu destroy` primeiro em `environments/prod` e depois em `environments/dev`.

Para refletir: que valores são definidos pelo módulo e quais são configurados por cada ambiente? Por que é importante não aplicar os dois ambientes a partir do mesmo diretório de trabalho?

Módulo 4 — Floci, S3, encriptação do estado e CI/CD

Objetivo: usar o provider AWS contra um emulador local, provisionar um bucket S3 e acompanhar o resultado numa página web.

1. Volte à raiz do repositório. Se seguiu os passos anteriores no mesmo terminal, está em `environments/prod`; regresse à raiz com:

```
cd ../../../
```

2. Inicie o Floci e a dashboard:

```
docker compose up --build -d
docker compose ps
```

3. Abra `http://localhost:8080`. O painel deve mostrar o estado da ligação ao Floci e informar que o bucket ainda não foi criado. O endpoint AWS local é `http://localhost:4566`.
4. Aceda ao módulo e leia `providers.tf` e `main.tf`. Confirme que o provider AWS aponta o endpoint S3 para o Floci e que o recurso declarado é um

bucket.

5. Inicialize, valide e reveja o plano:

```
cd modulo-4-avancado-cicd
tofu init
tofu fmt
tofu validate
tofu plan
```

6. Aplique a configuração e volte ao painel:

```
tofu apply
```

   Confirme em `http://localhost:8080` que o bucket `minicurso-opentofu-migrado` aparece na lista e está provisionado.

7. Reveja `.github/workflows/opentofu.yml`. O workflow prepara o OpenTofu, inicia o Floci, valida a configuração e gera um plano em pull requests. Em pushes para `main`, também aplica a configuração.

8. Quando terminar, remova o bucket e pare os serviços:

```
tofu destroy
cd ..
docker compose down
```

Para refletir: qual é a diferença entre apontar um provider para a AWS real e apontá-lo para o endpoint do Floci? O que a dashboard mostra antes e depois do `apply`?

> A passphrase de encriptação incluída no Módulo 4 é apenas ilustrativa. Não use passphrases ou credenciais embutidas no código em ambientes reais.

Resumo

Ao concluir, deverá conseguir explicar que OpenTofu:

- descreve recursos declarativamente em ficheiros HCL;

- usa providers para comunicar com plataformas ou serviços;
- calcula diferenças entre o estado atual e o desejado antes de aplicar;
- guarda estado para acompanhar recursos geridos;
- permite reutilizar configurações com módulos e separar ambientes;
- pode trabalhar com serviços emulados localmente, como o Floci, para praticar sem provisionar recursos numa conta AWS real.